

Distributed Systems

Ergebnisbericht Übung 2

Ziel dieser Übung war es, einen Schnappschussalgorithmus zu implementieren. Als Grundlage dafür diente eine simulierte verteilte Anwendung eines Banksystems, in dem mehrere Nodes vollständig miteinander vernetzt sind. In zufälligen Abständen sendet jede Node einer zufällig ausgewählten Node einen Betrag X. Dieser Betrag wird vor dem Versenden der Nachricht vom Konto der sendenden Node abgezogen und nach Erhalt der Nachricht auf das Konto der Empfangenden Node eingezahlt. Bei diesem Nachrichtenfluss können Inkonsistenzen auftreten, insbesondere dann, wenn der Zustand des Netzwerks gemessen wird während Transaktionen unterwegs sind.

Zum Start habe ich mir die Struktur des Projekts von Codex aufsetzen lassen. Die Einbindung des Bankingsystems in sim4da ging problemlos mit einer relativ kompakten Klasse. Im Wesentlichen initialisiert das Bankhaus alle zugehörigen Nodes und startet dann den Simulator für X Sekunden. Im engage-Loop einer Node passieren zwei relevante Dinge. Zum einen wird ein separater Thread gestartet, der die periodischen Transaktionen verwaltet. Der andere Part besteht aus dem while (True) Loop, der so lange auf empfangene Nachrichten hört, wie die Simulation läuft.

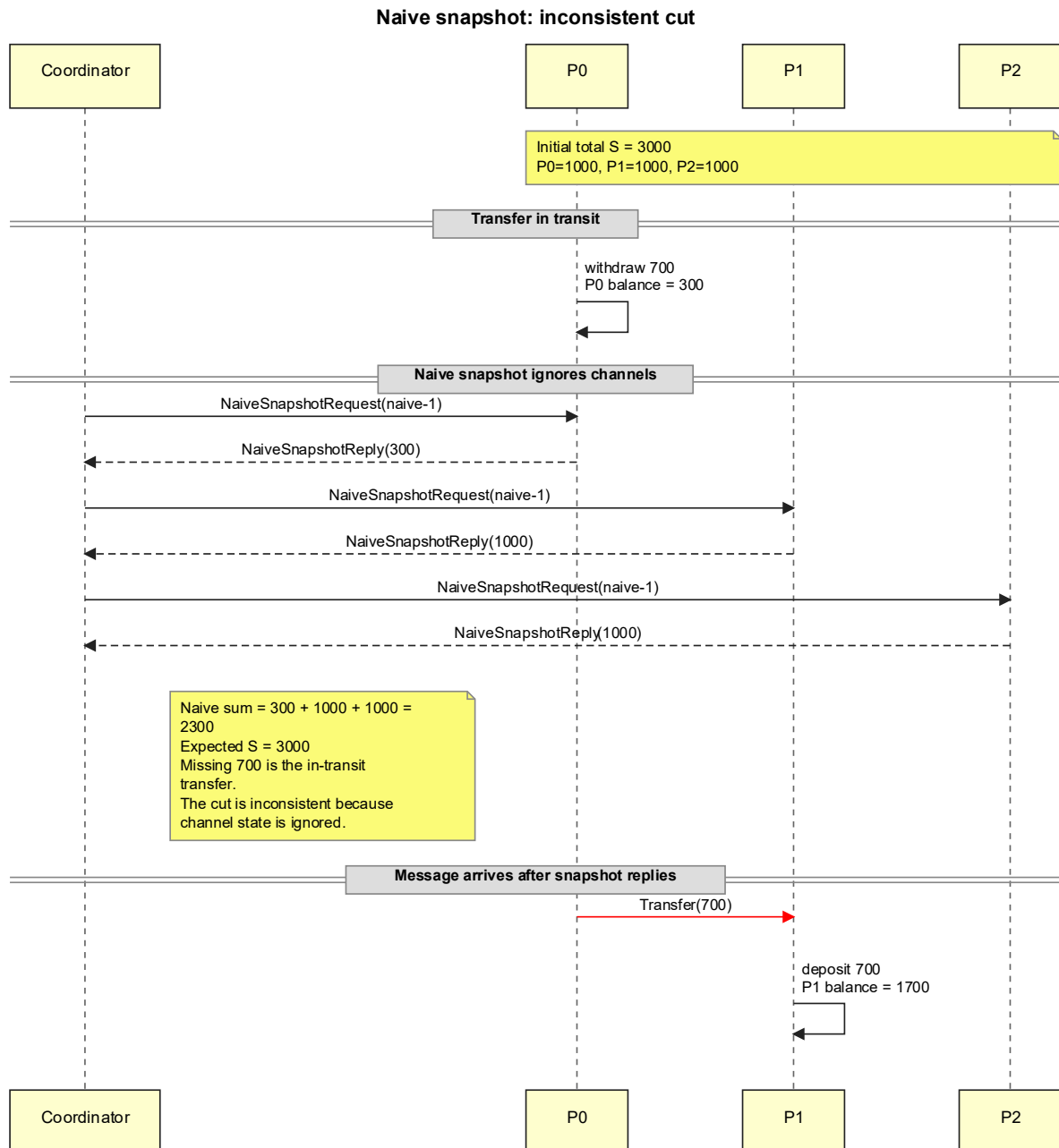
Beim Empfang einer Nachricht kann diese Nachricht verschiedene Typen haben. Der einfachste Fall ist eine Transaktion. Hierbei wird die Menge Geld vom Sender-Konto entgegengenommen und auf das eigene Konto gutgeschrieben. Wichtig zu beachten ist hierbei, dass die Zugriffe auf die eigene Balance sowohl beim Sender als auch beim Receiver synchronisiert werden müssen, damit nur ein Thread gleichzeitig auf den Kontostand zugreifen kann. Sonst könnte es passieren, dass die Node zur selben Zeit Geld versenden will, wie sie auch Geld empfängt, was zu einer Inkonsistenz führen würde.

Um nun einen konsistenten Schnappschuss des globalen Zustands des Systems zu erstellen, wurde ein Schnappschuss Koordinator gemäß Variante (a) implementiert. Der Koordinator verwaltet und regelt das gesamte Prozedere zum Erstellen eines Schnappschusses. Schnappschüsse werden nach fest definierten Zeitpunkten erstellt. Das Schnappschuss Verfahren nimmt an, dass keine Inkonsistenzen innerhalb einer Node entstehen (Bsp.: synchroner Zugriff auf Kontostand). Das Verfahren nimmt weiterhin an, dass die Anwendung ohne Crash terminiert und, dass die Anwendung keine Nachrichten verschwinden lässt. Bei Nachrichtenverlust könnte auch der Schnappschussalgorithmus keine zuverlässige Aussage mehr darüber treffen, ob der Zustand konsistent ist. Da die FiFo Eigenschaft nicht garantiert ist, habe ich mich für den Coloring Ansatz entschieden, um robust gegen die fehlende FiFo Reihenfolge zu sein.

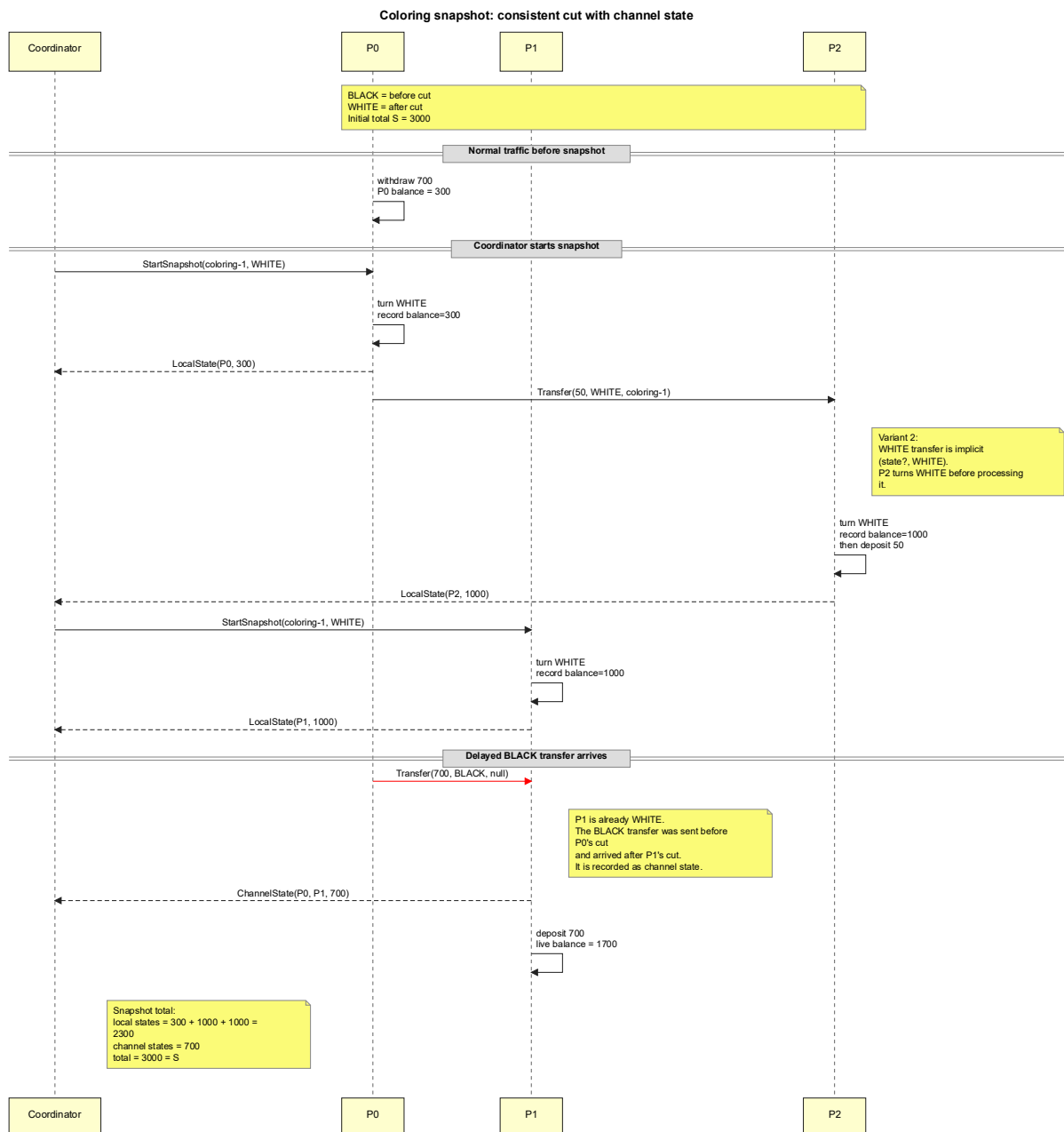
Zu Beginn der Simulation sind alle Nodes schwarz codiert, auch alle Transfers werden schwarz codiert. Wenn ein Schnappschuss angefragt wird, schickt der Koordinator eine Kontrollsequenz an alle Nodes. Jede Node notiert daraufhin ihren Kontostand codiert sich

selbst weiß und schickt den Kontostand an den Koordinator zurück. Zukünftige Transaktionen von dieser Node sind ab sofort weiß codiert. Transaktionen, die zuvor schwarz codiert versendet wurden bleiben schwarz. Da keine FiFo Reihenfolge garantiert ist, kann es natürlich vorkommen, dass ein schwarz codierter Prozess eine weiß codierte Transaktion erhält. In diesem Fall wird zuerst der Kontostand an den Koordinator übermittelt, danach wird der Inhalt der Transaktion ausgeführt. Der Schnappschuss bleibt dadurch konsistent. Eine weiß codierte Transaktion zu erhalten, bedeutet implizit, dass ein Schnappschuss vom Koordinator angefragt wurde. Dieser Fall kann eintreten, wenn die Schnappschuss Anfrage längere Zeit unterwegs ist, oder sogar verloren geht. Die Transaktion die unterwegs sind werden erkannt, wenn eine schwarz codierte Transaktion bei einer weiß codierten Node ankommt. Die Node erkennt, dass diese Transaktion aufgrund der schwarzen Codierung relevant für den Schnappschuss ist und meldet den Inhalt der Transaktion an den Koordinator. Die Transaktion wird nach dem Melden normal ausgeführt, um die Bankanwendung korrekt weiterlaufen zu lassen. Wenn alle State Meldungen gesammelt wurden, überprüft der Koordinator die Invariante und entscheidet über Konsistenz.

Fragt man mit einem naiven Schnappschuss lediglich die Nodes zum Zeitpunkt X, wie denn ihr Kontostand ist, so ist es möglich, dass unterwegs befindliche Nachrichten nicht mitgezählt werden und somit die Gesamtsumme des Geldes im Netzwerk nicht stimmt. Der Vergleich der beiden Schnappschussalgorithmen lässt sich gut anhand der folgenden UML-Diagramme erkennen:



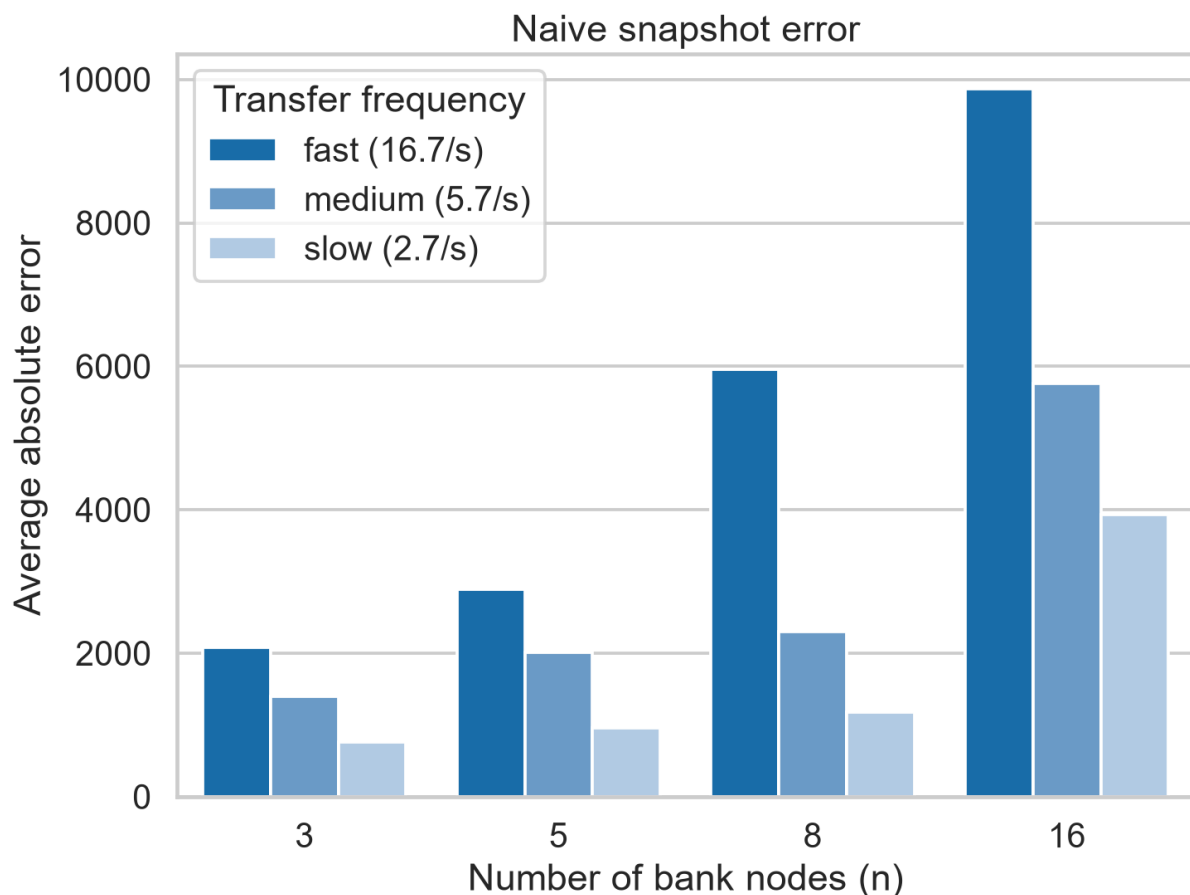
Node 0 versendet 700 Geldeinheiten und reduziert ihren Kontostand. Während diese Transaktion unterwegs ist, wird ein Schnappschuss erstellt. Da alle Nodes nur ihre Kontostände melden, fehlen 700 Geldeinheiten zur Erfüllung der Invariant und die Sicht auf den globalen Zustand ist inkonsistent.



In diesem Beispiel werden Nodes und Transaktionen codiert. Wie im Beispiel davor sendet Node 0 eine Transaktion mit 700 Geldeinheiten an Node 1. Diese Transaktion ist schwarz codiert. Während die Transaktion unterwegs ist, wird ein Schnappschuss angefragt. Node 0 wird weiß codiert, schickt danach aber noch eine Transaktion (jetzt weiß codiert) an Node 2. Node 0 meldet den Kontostand von 300 zurück an den Koordinator. Node 2 hat noch kein Schnappschuss Signal vom Koordinator erhalten, bekommt aber nun die weiß codierte Transaktion von Node 0. Daraufhin kann Node 2 schließen, dass sie sich nun auch weiß codieren muss und meldet als erstes den Kontostand an den Koordinator. Danach führt sie die eingehende Transaktion aus. Dasselbe Spiel für Node 1. Nachdem alle Nodes weiß codiert sind kommt jetzt noch die schwarz codierte Transaktion in Höhe von 700 Geldeinheiten bei Node 1 an. Da Node 1 aber schon weiß ist, kann sie schließen, dass die Transaktion zum Channel State gehört und daher an den Koordinator gemeldet werden muss. Nach dem Melden führt Node 1

die Transaktion aus. Der Schnappschuss ist konsistent, da er auch die Geldeinheiten aus der Transaktion zusätzlich zu den Kontoständen enthält.

Der Aufwand, der jedoch betrieben werden muss, um konsistente Sichten auf den globalen Zustand zu bekommen steigt mit der Zahl der Netzwerkteilnehmer und auch mit der Anforderung an Geschwindigkeit stark an. Zum Beispiel kann man deutlich erkennen, dass bei einer durchschnittlichen Transfergeschwindigkeit von ca. 16 Transaktionen/s deutlich höhere Fehler im Sinne der Inkonsistenz zustande kommen, wenn man den naiven Schnappschussalgorithmus verwendet. Auch wenn der Fehler mit langsamerer Frequenz abnimmt, steigt er dennoch stark mit der Anzahl an Netzwerkteilnehmern. Das liegt schlicht und ergreifend daran, dass mit mehr Netzwerkteilnehmern auch automatisch mehr Transaktionen gleichzeitig unterwegs sind, was dazu führt, dass mit dem naiven Schnappschussalgorithmus mehr Transaktionen nicht erfasst werden.



Betrachtet man den Koordinierten Schnappschuss so fällt auf, dass die Menge an benötigten Nachrichten zur Sicherstellung der Konsistenz stark zunimmt, je mehr Nodes im Netzwerk registriert sind. Auch höhere Transaktionsfrequenzen führen ab einem gewissen Punkt zu deutlich höherem Nachrichtenaufkommen. Das liegt vermutlich daran, dass mit höherer Transaktionsgeschwindigkeit mehr Transaktionen um den Schnappschuss herum anfallen, die alle mit einer Kontrollnachricht behandelt werden müssen.

